Friend Functions:

One of the important concepts of OOP is data hiding, i.e., a non-member function cannot access an object's private or protected data.

But, sometimes this restriction may force programmer to write long and complex codes. So, there is mechanism built in C++ programming to access private or protected data from non-member functions.

This is done using a friend function or/and a friend class.

If a function is defined as a friend function then, the private and protected data of a class can be accessed using the function.

The complier knows a given function is a friend function by the use of the keyword **friend**.

For accessing the data, the declaration of a friend function should be made inside the body of the class (can be anywhere inside class either in private or public section) starting with keyword `friend`.

Syntax :

```
class className
{
    ... .. ...
    friend return_type functionName(argument/s);
    ... .. ...
}

return_type functionName(argument/s)
{
    ... .. ...
    // Private and protected data of className can be accessed
from
    // this function because it is a friend function of
className.
    ... .. ...
}
```

Vector India

Example:

```
#include<iostream>
using namespace std;

class Distance

{

     int meter;

     public: Distance()

          {

               cout<<”enter the meter:”<<endl;

               cin>>meter;

          }

          void Display()

          {

               cout<<”Meter:”<<meter<<endl;

          }

friend int addfive(Distance); // making addfive function friend
to Distance class

     };

// friend function definition

int addfive(Distance D)

{    D.meter+=5;

     return D.meter;
```

Vector India

```
}
int main()
{
     Distance Dobj;
     Dobj.Display();
     cout<<addfive(Dobj)<<endl;
}
```

Output:

```
enter the meter :10
Meter : 10
15
```

Making Member function as a friend:

syntax :

```
friend return_type classname :: functionName(argument/s);
```

Example :

```
class XYZ;   // forward declaration
class ABC
{
     int data1;
     public: ABC()
               {
                    data1=10;
```

Vector India

```
                }
                void Add(XYZ);
};
class XYZ
{
     int data2;
     public: XYZ()
                {
                    data2=20;
                }
     friend void ABC :: Add(XYZ ); // making Add member function
of ABC class friend to XYZ class
};
void ABC:: Add(XYZ obj)
{
     cout<<data1+obj.data2<<endl;
}
main()
{
     ABC aobj;
     XYZ xobj;
     aobj.Add(xobj);
```

Vector India

```
}
```

friend class :

Similarly, like a friend function, a class can also be made a friend of another class using keyword friend. For example:

```
... .. ...

class B;
class A
{
   // class B is a friend class of class A
   friend class B;
   ... .. ...
}

class B
{
   ... .. ...
}
```

When a class is made a friend class, all the member functions of that class becomes friend functions.

In this program, all member functions of `class B` will be friend functions of `class A`. Thus, any member function of class B can access the private and protected data of class A. But, member functions of class A cannot access the data of class B.

Remember, friend relation in C++ is only granted, not taken.

Example:

```
#include <iostream>
using namespace std;

class Square;
```


Vector India

```
class Rectangle {
    int width, height;
  public:
    int area ()
      {return (width * height);}
    void convert (Square a);
};

class Square {
friend class Rectangle;//making Rectangle class friend to Square
class
  private:
    int side;
  public:
    Square (int a) : side(a) {}
};
// rectangle class member function accessing the private member
of Square class
void Rectangle::convert (Square a) {
  width = a.side;
  height = a.side;
}

int main () {
  Rectangle rect;
  Square sqr (4);
  rect.convert(sqr);
  cout << rect.area();
  return 0;
}
```

Vector India